GitHub Issues Connector – Full Local Debugging Narrative (Joe Summary)

Summary of Key Points

This document is a comprehensive, multi-page record of several days of local debugging work carried out on the GitHub Issues Connector (Microsoft Graph External Connector) using Azure Functions. The work was done without an Azure subscription, without a Microsoft 365 tenant, and entirely via Visual Studio Code and Azure Functions Core Tools.

The primary blockers were not TypeScript configuration, ports, or file locations, but rather Azure Functions startup lifecycle behavior, specifically: eager Azure Identity creation, module-load-time exceptions, and storage-dependent Timer triggers in Development. The final solution mirrors the PnP OAuth sample architecture by explicitly disabling Azure-dependent functionality during local development while preserving production wiring.

Background and Initial Context

The objective was to follow the Microsoft Learn module 'Build your first Microsoft 365 Copilot connector' while running the GitHub Issues Connector locally. Unlike the Learn prerequisites, this work intentionally proceeded without:

- an Azure subscription
- a Microsoft 365 tenant with Copilot enabled
- Teams publish capability

Instead, the goal was to understand the local runtime behavior, configuration flow, and architectural differences between a pure Azure Functions project and PnP / Teams Toolkit-driven samples.

Phase 1 – TenantId and Azure Identity Failures

The earliest and most persistent error was:

'ClientSecretCredential: tenantId is a required parameter'

This error occurred even though the compiled JavaScript files existed and Azure Functions Core Tools could locate the entry point. The error appeared as 'Unable to load entry point' and 'No job functions found', which initially suggested missing files or misconfigured paths.

Key realization: 'Unable to load entry point' in Azure Functions does NOT usually mean the file is missing; it almost always means a top-level exception occurred while evaluating the module.

Phase 2 – Why config.ts Alone Was Not Enough

Early mitigation efforts focused on config.ts: relaxing validation, skipping tenant/client checks in Development, and aligning environment variable usage with PnP samples. While correct, these changes alone did not resolve the startup crash.

Critical insight: Azure Identity was not being constructed in config.ts. It was being constructed in graphClient.ts, which is imported earlier in the startup chain. Because ClientSecretCredential was instantiated eagerly, the error occurred before any Development guards or HTTP handlers could run.

Phase 3 – graphClient.ts as the Decisive Fix

The decisive fix was to adopt the same implicit pattern used by the PnP OAuth samples: never instantiate Azure credentials during Development.

Key change:
- Replace 'throw new Error' with 'return null' when in Development
- Allow the Functions host to complete startup
- Fail only when Graph functionality is actually invoked

This change resolved the tenantId failure completely and allowed Azure Functions to progress to listener startup.

JoeDebug Terminology (Formalized)

JoeDebug2 – Launch Azure Functions_PORTABLE via func.exe start. This mode surfaces startup-time failures and is the authoritative way to debug Azure Functions locally.

JoeDebug3 – Attach-only Node debugger on port 9229. This mode assumes an already-running Functions host and was explicitly not required for this exercise.

Phase 4 – Timer Triggers and Azure Storage

After resolving identity issues, Azure Functions progressed further and exposed the next blocker:

'No connection could be made because the target machine actively refused it (127.0.0.1:10000)'

This error is produced when Timer triggers attempt to use Azure Storage. The presence of:
AzureWebJobsStorage = UseDevelopmentStorage=true
implicitly requires Azurite to be running locally. Without Azurite, the Functions host cannot complete listener startup and therefore never binds to HTTP port 7071.

/api Routing and Port Clarifications

The /api path in Azure Functions is virtual and injected by the host. There is never an /api folder on disk.

Port 7071 is the default HTTP port. 'localhost refused to connect' indicates that the Functions host never reached steady state due to binding failures, not that routing or port flags are missing.

Key Screenshots Referenced

The following screenshots were reviewed during debugging and are referenced conceptually in this document:

- VS Code showing dist/src/functions/pingJoe.js confirming successful build output
- Browser error 'localhost refused to connect' demonstrating host not binding to port 7071
- Terminal output showing Timer listener failure at 127.0.0.1:10000
- graphClient.ts with Joe-style Development guard returning null

(Screenshots are intentionally not embedded to preserve portability but are described precisely.)

Final State and Recommended Dev Posture

Recommended local development posture:
- Disable Timer triggers in Development
- Keep HTTP-only endpoints enabled (e.g., pingJoe)

- Skip Azure Identity and Graph calls locally
- Re-enable full Azure wiring only when a tenant and storage are available

This posture exactly matches the effective behavior of the PnP OAuth samples.

References

- Microsoft Learn: Build your first Microsoft 365 Copilot connector
- Azure Functions Node.js runtime lifecycle documentation
- PnP Copilot pro-dev samples (OAuth-off Development pattern)